

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

***CO1:** Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr Ramamoorthy

QUESTIONS: 6

Total Marks:30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

*I NIRANJANA A(192521370) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

NIRANJANA A

Short Answer Test

1. Smart Banking System

A Smart Banking System allows customers to open accounts, transfer funds, and pay utility bills online.

(a) Object-Oriented Concepts

Class: A class is a blueprint used to create objects. It defines the attributes and methods of an object.

Example: Customer, Account, Transaction.

Object: An object is an instance of a class. It contains data and performs actions.

Example: Customer "Niranjana" with Account No. 123456789.

Message Passing: Objects communicate by calling methods.

Example: A Customer object sends a request to the Account object to transfer funds.

Encapsulation: Encapsulation combines data and methods into a single unit and protects data from unauthorized access.

Example: Account balance is private and accessed only through methods like deposit(), withdraw(), and getBalance().

Polymorphism: Polymorphism allows one method to perform different tasks depending on the object.

Example: calculateInterest() calculates different interest for Savings, Fixed Deposit, and Loan accounts.

(b) Object Relationships

Association: Two objects work together but exist independently.

Example: Customer uses an Account.

Aggregation: One object contains another, but both can exist independently.

Example: A Bank has many Customers.

Composition: One object cannot exist without another.

Example: Transaction History belongs to an Account. If the account is deleted, the transaction history is also deleted.

(c) Software Reusability and Maintainability

- Reusable classes reduce code duplication.
 - Encapsulation protects sensitive data.
 - Polymorphism supports different account types.
 - Modular design makes testing and maintenance easier.
 - The banking system becomes secure, scalable, and easy to maintain.
-

2. Smart Healthcare Management System

A Smart Healthcare Management System manages patients, doctors, appointments, and medical records.

(a) Object-Oriented Design Principles

Abstraction: Shows only essential features while hiding implementation details.
Example: A patient books an appointment without knowing the internal process.

Encapsulation: Protects patient data by keeping it private.
Example: Medical records are accessed only through authorized methods.

Modularity: Divides the system into independent modules.
Example: Patient Management, Appointment, Billing, and Pharmacy modules.

(b) SOLID Principles

Liskov Substitution Principle (LSP): A child class should replace its parent class without affecting program correctness.
Example: GeneralDoctor and SpecialistDoctor can both replace the Doctor class.

Interface Segregation Principle (ISP): Classes should implement only the methods they need.
Example: Separate interfaces for Appointment, Billing, and Prescription services.

(c) Software Quality and Flexibility

- Abstraction reduces complexity.
- Encapsulation improves security.
- Modularity simplifies maintenance.
- LSP ensures reliable inheritance.
- ISP keeps code clean and flexible.
- The healthcare system becomes secure, scalable, and maintainable.

3. Online Food Delivery System

An Online Food Delivery System allows customers to order food online and track deliveries.

(a) SDLC Phases

Planning: Define project goals, budget, and timeline.

Requirement Analysis: Gather customer requirements such as ordering, payment, and delivery tracking.

Design: Design the database, user interface, and system architecture.

Development: Write program code and implement features.

Testing: Identify and fix software errors.

Maintenance: Update software, fix bugs, and add new features after deployment.

(b) Purpose of Each Phase

- Planning – Defines project scope.
- Requirement Analysis – Collects user needs.
- Design – Creates system architecture.
- Development – Builds the software.
- Testing – Ensures quality.
- Maintenance – Keeps software updated.

(c) Contribution of SDLC

- Provides systematic development.
- Reduces development errors.
- Improves software quality.
- Makes software secure and reliable.
- Simplifies future maintenance.

4. Smart Agriculture Monitoring System

A Smart Agriculture Monitoring System monitors crops, soil, temperature, and irrigation using IoT sensors.

(a) SDLC Models

Prototype Model: Builds a working prototype to gather user feedback before final development.

Spiral Model: Combines development with risk analysis through repeated cycles.

Agile Model: Develops software in small iterations with continuous customer feedback.

DevOps Model: Integrates development and operations for continuous testing and deployment.

(b) Prototype Model vs Agile Model

Prototype Model	Agile Model
Builds a prototype first.	Develops in iterations.
Used when requirements are unclear.	Used when requirements change frequently.
Feedback on prototype.	Feedback after every iteration.
Final system developed after prototype approval.	Continuous improvement throughout development.

(c) Recommended Model

Agile Model

Justification:

- Supports changing requirements.
- Allows frequent customer feedback.
- Delivers software quickly.
- Supports continuous testing.
- Easy to add new features.

5. Smart Transport Management System

A Smart Transport Management System manages vehicles, routes, bookings, and ticketing.

(a) Importance of UML

- Provides a visual representation of the system.
- Improves communication.
- Helps understand system structure.
- Detects design problems early.
- Makes maintenance easier.

(b) UML Diagrams

State Diagram: Shows object states and transitions.

Example: Booked → Confirmed → Completed.

Component Diagram: Shows software components and their relationships.

Example: Booking, Payment, Database, Tracking.

Deployment Diagram: Shows hardware and software deployment.

Example: Mobile App, Server, Database, GPS Device.

Package Diagram: Organizes classes into packages.

Example: User, Booking, Payment, Reports.

(c) Benefits of UML

- Improves communication among stakeholders.
 - Reduces misunderstandings.
 - Simplifies system documentation.
 - Supports better maintenance.
 - Produces high-quality software.
-

6. Smart Inventory Management System

A Smart Inventory Management System manages products, stock, suppliers, and orders.

(a) Modular Software Design

Modular software design divides the system into independent modules.

Modules:

- Product Management
- Stock Management
- Supplier Management
- Order Management
- Report Generation

Each module performs a specific task independently.

(b) Procedural Programming vs Object-Oriented Programming

Procedural Programming	Object-Oriented Programming
Divided into functions.	Divided into classes and objects.
Focuses on procedures.	Focuses on objects.
Less secure.	Uses encapsulation for security.
Limited code reuse.	Supports inheritance and polymorphism.
Difficult to maintain large projects.	Easier to maintain and extend.

(c) Benefits of Modularity

Scalability: New modules can be added without affecting existing modules.

Maintainability: Errors can be fixed in individual modules, making debugging easier.

Reusability: Existing modules can be reused in other software, reducing development time.

Conclusion :Modular software design and object-oriented programming make the Smart Inventory Management System scalable, maintainable, reusable, secure, and easy to develop.

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps

